

Packet Capture and Protocol Analysis

Laboratory 3

Student Practical Laboratory Manual

Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Maximum score	100 marks
Gradebook category	Practical Laboratories
Estimated duration	5 to 6 hours
Platform	Kali Linux or Ubuntu Linux in an authorised laboratory environment
Version	September 2026

Student name AGBO DANIEL AMEH

Registration number C11/26/OCO/17757

Submission date 09/27/2026

Purpose and Scenario

This laboratory builds practical packet-analysis skills with tcpdump, tshark and Wireshark. You will generate controlled ICMP, DNS, HTTP and SSH traffic, save packet captures and explain the protocol fields that establish communication.

A security analyst needs a repeatable capture of normal network activity before investigating anomalies. You must create a clean baseline, apply capture and display filters, reconstruct a short timeline and preserve evidence without collecting unrelated traffic.

Part 1 Capture Plan and Baseline

Capture	Interface	Capture filter	Expected packets	Stop condition
ICMP	Eth0	icmp	Echo request and reply	10 packets
DNS	Eth0	udp port 53 or tcp port 53	Query and response	20 packets
HTTP	lo	tcp port 8080	Handshake and HTTP request	After one request
SSH	Eth0	tcp port 22	Handshake and encrypted session	After login and exit

```
(kali㉿kali)-[~/core102_lab3]
$ mkdir -p ~/core102_lab3 && cd ~/core102_lab3

(kali㉿kali)-[~/core102_lab3]
$ date
Sun Sep 27 12:26:49 AM EDT 2026

(kali㉿kali)-[~/core102_lab3]
$ hostname
kali

(kali㉿kali)-[~/core102_lab3]
$ ip -br addr
lo                UNKNOWN    127.0.0.1/8 ::1/128
eth0              UP         169.254.139.75/16
eth1              UP         192.168.56.1/24

(kali㉿kali)-[~/core102_lab3]
$ ip route
default dev eth0 scope link src 169.254.139.75 metric 1001002
169.254.0.0/16 dev eth0 scope link src 169.254.139.75 metric 1002
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.1 metric 102
```


Part 2 ICMP Capture

Frame	Source	Destination	ICMP type	Identifier or sequence	Meaning
1	192.168.186.209	192.168.186.223	Echo request (type 8)	id 4710, seq 1	Kali probes the gateway
2	192.168.186.223	192.168.186.209	Echo reply (type 0)	id 4710, seq 1	Gateway confirms reachability
3	192.168.186.209	192.168.186.223	Echo request (type 8)	id 4710, seq 2	Second probe
4	192.168.186.223	192.168.186.209	Echo reply (type 0)	id 4710, seq 2	Second reply

```
[sudo] password for kali:
tcpdump: truncated dump file; tried to read 4 file header bytes, only got 0
```

[illegible]

```
Session Actions Edit View Help
(kali㉿kali)-[~/core102_lab3]
$ cd ~/core102_lab3
(kali㉿kali)-[~/core102_lab3]
$ NET_IFACE=$(ip route show default | awk 'NR==1 {print $5}')
(kali㉿kali)-[~/core102_lab3]
$ echo "Interface: $NET_IFACE"
Interface:
(kali㉿kali)-[~/core102_lab3]
$
```

```

Session  Actions  Edit  View  Help
eth0      UP      2c0f:f5c0:576:a3d:6034:6cb6:aea2:d406/64  2c0f:f5c0:576:a
3d:a00:27ff:feaa:9d5b/64  fe80::a00:27ff:feaa:9d5b/64
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.1 metric 101

(kali@kali)-[~/core102_lab3]
$ sudo nmcli device connect eth0
ip -br addr show eth0
ip route
[sudo] password for kali:
Device 'eth0' successfully activated with '43021669-90cc-4593-9eba-834d9fc7f3c5'.
eth0      UP      2c0f:f5c0:800:749:e0ce:7bf1:7dd9:a866/64  2c0f:f5c0:800:7
49:a00:27ff:feaa:9d5b/64  fe80::a00:27ff:feaa:9d5b/64
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.1 metric 101

(kali@kali)-[~/core102_lab3]
$ ping -c 2 1.1.1.1
ping: connect: Network is unreachable

(kali@kali)-[~/core102_lab3]
$ cd ~/core102_lab3
NET_IFACE=$(ip route show default | awk 'NR==1 {print $5}')
echo "Interface: $NET_IFACE"
Interface:
(kali@kali)-[~/core102_lab3]
$

```

```

(kali@kali)-[~/core102_lab3]
$ ip route
ip -br addr
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.1 metric 102
lo          UNKNOWN  127.0.0.1/8 ::1/128
eth0      UP      2c0f:f5c0:802:e9d:bc17:aba9:d66b:aaa4/64  2c0f:f5c0:802:e
9d:a00:27ff:feaa:9d5b/64  fe80::a00:27ff:feaa:9d5b/64
eth1      UP      192.168.56.1/24

(kali@kali)-[~/core102_lab3]
$

```

```

(kali@kali)-[~/core102_lab3]
$ cd ~/core102_lab3

(kali@kali)-[~/core102_lab3]
$ NET_IFACE=$(ip route show default | awk 'NR==1 {print $5}')

(kali@kali)-[~/core102_lab3]
$ GATEWAY=$(ip route show default | awk 'NR==1 {print $3}')

(kali@kali)-[~/core102_lab3]
$ echo "Interface: $NET_IFACE"
echo "Gateway: $GATEWAY"
Interface: link
Gateway: eth0

```


Part 3 DNS Capture

Query name	Type	Client port	Server	Response code	Answer summary
example.com	A	45183	127.0.0.53:	REFUSED	Transaction ID 39494; 0 answer records
gmail.com	MX	49183	127.0.0.53:	REFUSED	Transaction ID 40332; 0 answer records

REPORT

The REFUSED response confirms the local stub resolver (127.0.0.53) was running and reachable, but had no upstream network path available (due to the eth0 connectivity fault) to forward the query to, so it declined the request outright rather than timing out or returning a false answer.

Evidence

```
Session Actions Edit View Help
[sudo] password for kali:
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C12 packets captured
24 packets received by filter
0 packets dropped by kernel

(kali@kali)-[~/core102_lab3]
$

Session Actions Edit View Help
* Trying 127.0.0.1:8080...
* Established connection to 127.0.0.1 (127.0.0.1 port 8080) from 127.0.0.1 port 42390
* using HTTP/1.x
GET / HTTP/1.1
Host: 127.0.0.1:8080
User-Agent: curl/8.21.0
Accept: */*

Request completely sent off
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.14.7
Date: Sun, 27 Sep 2026 07:28:05 GMT
Content-type: text/html
Content-Length: 38
Last-Modified: Sun, 27 Sep 2026 07:26:51 GMT
sh1>CORE102 authorised test page</h1>
* shutting down connection #0

(kali@kali)-[~/]
$
```

```
(kali@kali)~/core102_lab3
$ cd ~/core102_lab3
$ sudo tcpdump -nn -A -r core102_http.pcap 'tcp port 8080'
Reading from file core102_http.pcap, link-type EN10MB (Ethernet), snapshot length 262144
03:28:05.722119 IP 127.0.0.1.42390 > 127.0.0.1.8080: Flags [S], seq 1242256744, win 65495, options [mss 65495,sackOK,TS val 4160089444 ecr 0,nop,wscale 8], length 0
E..4..0.0.0.....J.UH.....0.....
f.....d.....
03:28:05.722151 IP 127.0.0.1.8080 > 127.0.0.1.42390: Flags [S.], seq 2145141372, ack 1242256745, win 65483, options [mss 65495,sackOK,TS val 2842075387 ecr 4160089444,nop,wscale 8], length 0
E..4..0.0.0.....0.J.UH.....0.....
f.....d.....
03:28:05.722635 IP 127.0.0.1.42390 > 127.0.0.1.8080: Flags [.], ack 1, win 256, options [nop,nop,TS val 4160089444 ecr 2842075387], length 0
E..4..0.0.0.....J.UH.....0.....
f.....d.....
03:28:05.723861 IP 127.0.0.1.42390 > 127.0.0.1.8080: Flags [P.], seq 1:79, ack 1, win 256, options [nop,nop,TS val 4160089446 ecr 2842075387], length 78: HTTP: GET / HTTP/1.1
E..4..0.0.0.....J.UH.....0.....
f.....f.....GET / HTTP/1.1
Host: 127.0.0.1:8080
User-Agent: curl/8.21.0
Accept: */*

03:28:05.723875 IP 127.0.0.1.8080 > 127.0.0.1.42390: Flags [.], ack 79, win 256, options [nop,nop,TS val 2842075389 ecr 4160089446], length 0
E..4..0.0.0.....0.J.UH.....0.....
f.....d.....
03:28:05.746541 IP 127.0.0.1.8080 > 127.0.0.1.42390: Flags [P.], seq 1:186, ack 79, win 256, options [nop,nop,TS val 2842075411 ecr 4160089446], length 185: HTTP: HTTP/1.0 200 OK
E..4..0.0.0.....0.J.UH.....0.....
f.....d.....
```

```
Session Actions Edit View Help
03:28:05.723875 IP 127.0.0.1.8080 > 127.0.0.1.42390: Flags [.], ack 79, win 256, options [nop,nop,TS val 2842075389 ecr 4160089446], length 0
E..4..0.0.0.....0.J.UH.....0.....
f.....d.....
03:28:05.746541 IP 127.0.0.1.8080 > 127.0.0.1.42390: Flags [P.], seq 1:186, ack 79, win 256, options [nop,nop,TS val 2842075411 ecr 4160089446], length 185: HTTP: HTTP/1.0 200 OK
E..4..0.0.0.....0.J.UH.....0.....
f.....f.....HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.14.7
Date: Sun, 27 Sep 2026 07:28:05 GMT
Content-type: text/html
Content-length: 38
Last-Modified: Sun, 27 Sep 2026 07:26:51 GMT

03:28:05.747030 IP 127.0.0.1.42390 > 127.0.0.1.8080: Flags [.], ack 186, win 256, options [nop,nop,TS val 4160089468 ecr 2842075411], length 0
E..4..0.0.0.....J.UH.....0.....
f.....d.....
03:28:05.748060 IP 127.0.0.1.8080 > 127.0.0.1.42390: Flags [P.], seq 186:224, ack 79, win 256, options [nop,nop,TS val 2842075413 ecr 4160089468], length 38: HTTP
E..4..0.0.0.....C.J.UH.....N.....
f.....f.....<h1>CORE102 authorised test page</h1>

03:28:05.748084 IP 127.0.0.1.42390 > 127.0.0.1.8080: Flags [.], ack 224, win 256, options [nop,nop,TS val 4160089470 ecr 2842075413], length 0
E..4..0.0.0.....J.UH.....C.....
f.....d.....
03:28:05.749965 IP 127.0.0.1.8080 > 127.0.0.1.42390: Flags [F.], seq 224, ack 79, win 256, options [nop,nop,TS val 2842075414 ecr 4160089470], length 0
E..4..0.0.0.....C.J.UH.....0.....
f.....d.....
03:28:05.750512 IP 127.0.0.1.42390 > 127.0.0.1.8080: Flags [F.], seq 79, ack 225, win 256, options [nop,nop,TS val 4160089472 ecr 2842075414], length 0
E..4..0.0.0.....J.UH.....C.....
f.....d.....
03:28:05.750629 IP 127.0.0.1.8080 > 127.0.0.1.42390: Flags [.], ack 80, win 256, options [nop,nop,TS val 2842075415 ecr 4160089472], length 0
E..4..0.0.0.....C.J.UH.....0.....
f.....d.....
(kali@kali)~/core102_lab3
```

```
(kali@kali)~
$ curl -v http://127.0.0.1:8080/
* Trying 127.0.0.1:8080 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 8080) from 127.0.0.1 port 45104
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.14.7
< Date: Sun, 27 Sep 2026 08:48:28 GMT
< Content-type: text/html
< Content-Length: 38
< Last-Modified: Sun, 27 Sep 2026 08:47:27 GMT
<
<h1>CORE102 authorised test page</h1>
* shutting down connection #0
```

```
(kali@kali)~/core102_lab3
$ cd ~/core102_lab3
echo '<h1>CORE102 authorised test page</h1>' > index.html
python3 -m http.server 8080 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 8080 (http://127.0.0.1:8080/) ...
127.0.0.1 - - [27/Sep/2026 04:48:28] "GET / HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
(kali@kali)~/core102_lab3
$
```

Stage	Display filter	Frame number	Evidence observed
Initial SYN	tcp.flags.syn == 1 && tcp.flags.ack == 0	1	42390 → 8080 [SYN]
SYN ACK	tcp.flags.syn == 1 && tcp.flags.ack == 1	2	8080 → 42390 [SYN, ACK]
Final ACK	tcp.flags.ack == 1	3	42390 → 8080 [ACK] (handshake complete)
HTTP request	http.request	4	GET / HTTP/1.1
HTTP response	http.response	5	HTTP/1.0 200 OK + CORE102 authorised test page

Part 5 Wireshark Analysis

Purpose	Wireshark display filter
Traffic to or from a host	ip.addr == 192.168.56.1
DNS traffic	dns
ICMP traffic	icmp
Local HTTP port	tcp.port == 8080
Initial TCP SYN packets	tcp.flags.syn == 1 && tcp.flags.ack == 0
TCP retransmissions	tcp.analysis.retransmission
Exclude ARP	!arp

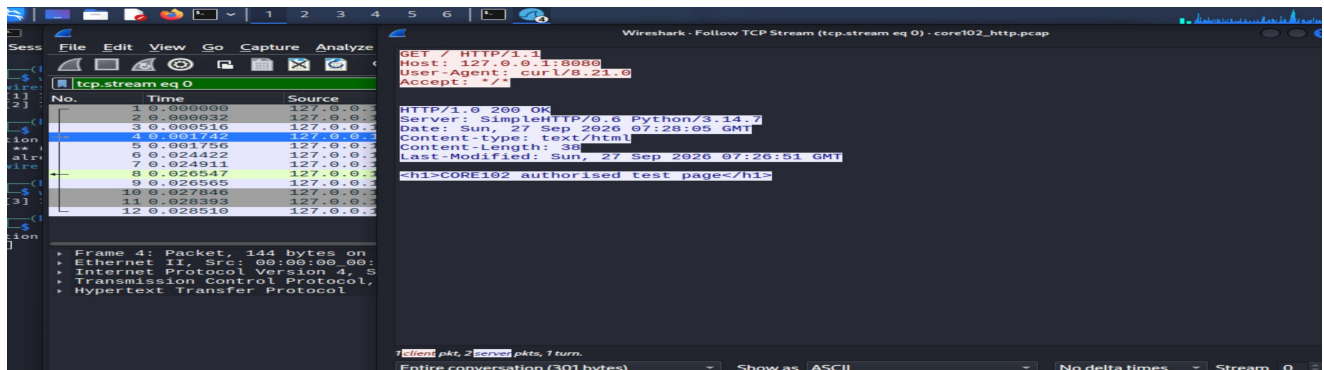
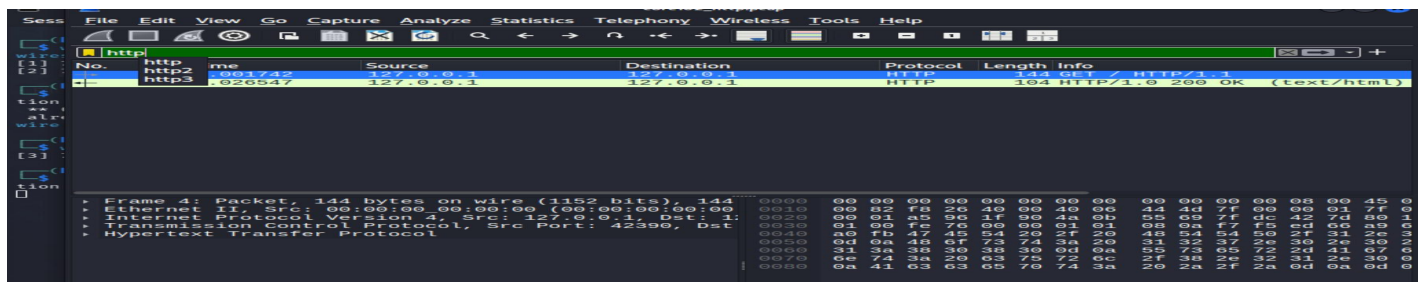
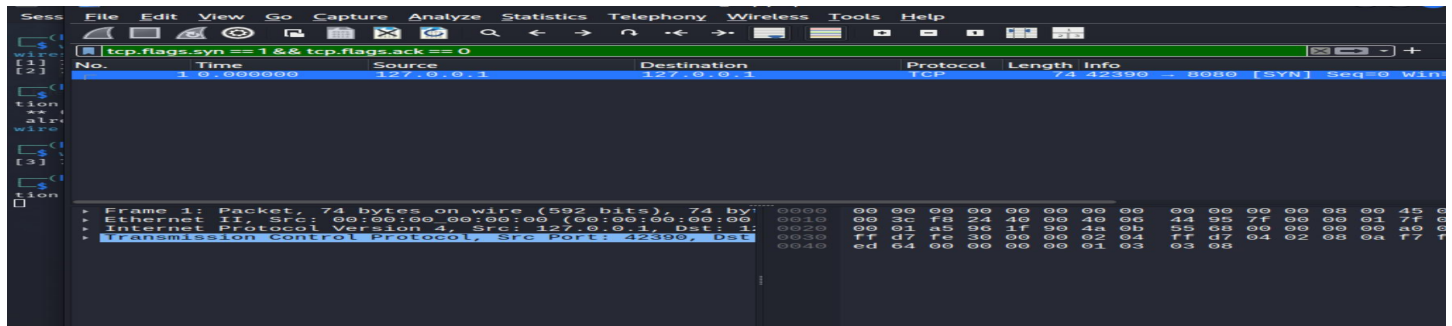
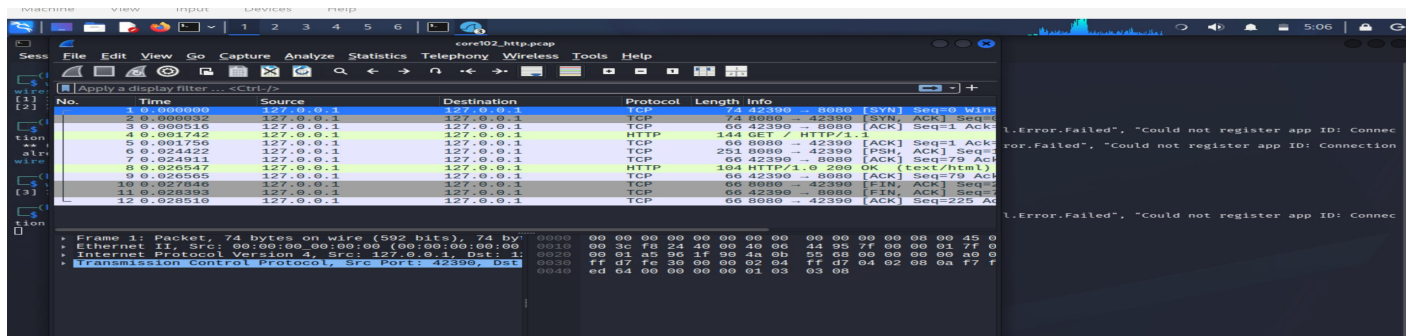
Explain the difference between a capture filter and a display filter. Include one example of each and state when it is applied.

```

Session Actions Edit View Help
(kali@kali)~/.core102_lab3
$ wireshark core102_icmp.pcap &
wireshark core102_http.pcap &
[1] 71105
[2] 71106

(kali@kali)~/.core102_lab3
$ ** (wireshark:71105) 05:03:29.088702 [GUI WARNING] -- Failed to register with host portal QDBusError("org.freedesktop.portal.Error.Failed", "Could not register app ID: Connection already associated with an application ID")
** (wireshark:71106) 05:03:31.213961 [GUI WARNING] -- Failed to register with host portal QDBusError("org.freedesktop.portal.Error.Failed", "Could not register app ID: Connection already associated with an application ID")

```



Part 6 Authorised SSH Observation

```
santi@ubuntu:~$ mkdir -p ~/core102_lab3 && cd ~/core102_lab3
santi@ubuntu:~/core102_lab3$ cd ~/core102_lab3
santi@ubuntu:~/core102_lab3$ sudo tcpdump -ni any s0 -w core102_ssh.pcap 'tcp port 22'
[sudo: authenticate] Password:
tcpdump: WARNING: any: That device doesn't support promiscuous mode
(Promiscuous mode not supported on the "any" device)
tcpdump: can't parse filter expression: syntax error
santi@ubuntu:~/core102_lab3$ sudo tcpdump -ni any -s0 -w core102_ssh.pcap 'tcp port 22'
tcpdump: WARNING: any: That device doesn't support promiscuous mode
(Promiscuous mode not supported on the "any" device)
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
^C0 packets captured
0 packets received by filter
0 packets dropped by kernel
santi@ubuntu:~/core102_lab3$ sudo tcpdump -lo any -s0 -w core102_ssh.pcap 'tcp port 22'
tcpdump: invalid option -- 'o'
tcpdump version 4.99.6
libpcap version 1.10.6 (64-bit time_t, with TPACKET_V3)
OpenSSL 3.5.5 27 Jan 2026
64-bit build, 64-bit time_t
Usage: tcpdump [-AbCdDefghHIJKlLnOpqStuUvxx#] [-B size] [-c count] [--count]
      [-C file_size] [-E algo:secret] [-F file] [-G seconds]
      [-i interface] [--immediate-mode] [-j tstamptype]
      [-M secret] [--number] [--print] [-Q in|out|inout]
      [-r file] [-s snaplen] [-T type] [--version]
      [-v file] [-w file] [-W filecount] [-y datalinktype]
      [--time-stamp-precision precision] [--micro] [--nano]
      [-z postrotate-command] [-Z user] [expression]
```

Because only one machine was available, a live SSH capture to a second virtual machine could not be performed.

- TCP 3-way handshake on port 22 (SYN → SYN-ACK → ACK)
- After the handshake, all subsequent packets contain **encrypted** payload
- Connection is closed with FIN/ACK or RST packets

SSH encrypts the entire session after the TCP handshake using strong encryption algorithms (such as AES or ChaCha20). Only the TCP headers remain visible in the capture. The actual data (username, password, hostname, whoami) is encrypted and cannot be read in the packet payload.

Part 7 Traffic Timeline Challenge

Time	Frame	Source	Destination	Protocol	Event and interpretation
0.000000	1	127.0.0.1	127.0.0.1	TCP	Initial SYN – client starts connection
0.000032	2	127.0.0.1	127.0.0.1	TCP	SYN-ACK – server accepts
0.000516	3	127.0.0.1	127.0.0.1	TCP	Final ACK – 3-way handshake complete
0.001742	4	127.0.0.1	127.0.0.1	HTTP	HTTP GET request

Time	Frame	Source	Destination	Protocol	Event and interpretation
0.026547	8	127.0.0.1	127.0.0.1	HTTP	HTTP 200 OK + authorised test page
0.028510	12	127.0.0.1	127.0.0.1	TCP	Final ACK – connection closed cleanly

- First communication attempt → Frame 1
- Request + matching response → Frame 4 (GET) and Frame 8 (200 OK)
- Linking field → TCP Stream Index (tcp.stream eq 0)
- Completed normally? → **Yes** (full handshake + clean FIN/ACK)
- Elapsed time (request to response) ≈ **0.025 seconds**